

HOW DOES DIFFICULTY AND PRIOR EXPERIENCE IMPACT SOLUTION QUALITY, TIME, AND PROMPTS IN AI-ASSISTED CODING?

Candidate 137
University of Bergen
Bergen, Norway

Candidate 134
University of Bergen
Bergen, Norway

Candidate 158
University of Bergen
Bergen, Norway

Candidate 104
University of Bergen
Bergen, Norway

Candidate 114
University of Bergen
Bergen, Norway

Abstract

Using AI to solve coding tasks has become prevalent. This raises concerns that programmers give up problem-solving and self-sufficiency to AI tools. The effects of this on future AI-assisted programming remains uncertain. While previous studies have examined how programmers use AI in coding, and others have discussed the evolution of AI-collaborative programming, there is limited knowledge about how AI-assistive programming affects self-sufficiency and problem-solving across experience levels. This paper investigates how users at three different skill levels interact with ChatGPT, the AI dependency and to which extent their skill level influences prompting strategies and solution quality. A workshop was conducted with eight participants, in addition with two participants at a later date, where the participants completed two separate programming tasks with full freedom to use ChatGPT. Afterwards, they completed a questionnaire to provide insight into their prompting behavior and experience interacting with ChatGPT. The findings from the workshop show that novices and experienced programmers use AI differently in their prompting strategies, level of reliance, and willingness to refine generated code. Based on these findings, we propose a more adaptable AI guidance, as well as improved prompting skills for programmers.

CCS Concepts

• **Human-centered computing** → **Empirical studies in HCI**; *Collaborative and social computing design and evaluation methods.*

Keywords

AI-assisted programming, ChatGPT, Prompting strategies, Human-AI collaboration, Empirical Study, Programming tasks

1 Introduction

The use of AI permeates modern programming. AI is used as a tool for problem solving and for direct support in writing code. Large language models, like ChatGPT and OpenAI's Codex, are widely accessible and capable of transforming natural language prompts into executable code, making them powerful tools for code generation and assistive programming. The access to such

tools marks a noticeable divide between programmers learning coding before widespread use of AI and those learning with AI [1]. This divide has led to raising concerns about the coding skills and understanding of novice programmers [1, 5–7]. Programmers today might be more reliant on AI to solve and debug code. The way they use AI might act as a replacement for manual skill and problem solving strategies [1]. Additionally, the ways in which programmers use AI for coding remains underexplored, despite their potential impact on solution quality, learning, and experience using AI. In this paper, we use the term AI-assistive programming to refer to the use of AI in programming - including both code generation and guidance.

As AI continues to develop rapidly, and becomes more sophisticated, the way people use AI and how this use will develop in the future is an important area of discussion. Previous studies have examined how programmers of different demographics use AI to code and what effect this has on learning outcomes [1, 6, 7]. While others have employed a more speculative perspective on human-AI coding collaboration. The future of programming may be more focused on interpreting, understanding and tweaking AI-generated code [5]. The notion of co-performance considers how humans can collaborate with artifacts to create and learn. Rather than AI autonomously producing solutions, there is value found in the complimentary capabilities of humans and AI [8].

While existing studies cover much of the field, several areas remain underexplored, particularly ChatGPT usage, prompting strategies across skill levels and task difficulty, and user experience. This paper examines how programmers at three skill levels use ChatGPT to solve Python tasks of varying difficulty, and how much they rely on or trust it. We ran a workshop with participants of three experience levels, evaluating solution quality, time spent, and prompting behavior. All participants were required to use ChatGPT but could choose how. Novices relied heavily on ChatGPT, using many specific prompts and rarely refining the generated code. Experienced programmers relied less, using broader prompts and treating ChatGPT as an assistive tool, refining its output to match their own ideas.

1.1 Motivation

The students writing this paper have developed their coding skills during a time when AI is widely accessible. Therefore, there is a desire to examine the methods in which other programmers utilize AI, and how this potentially can be improved. This paper examines AI-collaboration as a norm in contemporary coding, and presents



November 24, Bergen, Norway

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

the different collaborative methods people use to solve coding tasks. It provides HCI insight into prompting behavior, AI-reliance and agency, trust in AI, and motivation to solve problems. From the results, as students in IT, we will present and reflect over the current state of AI-assistive coding and make direct inferences on potential improvements for AI-usage in the future, which maximizes the technology's potential while promoting learning and self regulation.

2 Background

The appearance of powerful Large Language Models has triggered a shift in software development and computer science education. AI tools are being used as an essential part of programmers' daily workflow. The quick integration of LLMs into coding practices has dramatically changed how programmers learn and solve problems. Previous research documents the opportunities and risks that entail the use of AI-assisted programming. Studies show that AI models, such as ChatGPT, Copilot, and similar tools help programmers with productivity, reduce the effort required for repetitive coding tasks, code understanding, and learning new concepts. A study conducted by A. Bucaioni et al. provides evidence that AI and particularly ChatGPT, has the ability to solve coding tasks of easy to medium difficulty but struggles with complex tasks [4]. Other evidence suggests that AI tools improve student performance in programming courses through faster completion and better scores than those who did not, although the study did not find evidence of conceptual understanding and ease of learning. [1]. The implementation of AI tools had positive outcomes, but results varied across different studies, and it was emphasized that implementing AI tools requires carefully designed strategies to maximize benefits while ensuring that risks, such as over reliance and worse problem solving were mitigated [1].

Other studies on programming behaviour suggest that there are other important aspects of how skill influences the use of AI-assisted programming. Barke et al.'s Copilot study proposes two main interaction modes: acceleration and exploration, and how the level of the programmer often determines how much time they spend in each mode [2]. In acceleration mode, AI is used to speed up coding that they understand, while exploration is used to gain understanding and explore alternative options. The study shows that experienced programmers use more time in acceleration, use minimal prompts and are critical to coding suggestions from Copilot. On the other hand, less experienced programmers spend more of their time in exploration, write longer prompts and tend to trust responses more blindly [2]. Concerns of over-reliance and misplaced trust are further supported by Perry et al. and their study emphasizes that experience plays an important role, especially regarding the writing of secure code [10].

Finally, Jonson et al.'s study "the end of programming as we know it" argues that AI is reshaping the nature of programming from writing code towards a process where users instead must articulate intentions and interpret output [5]. This shift from writing to understanding code requires users to check and debug AI-created code, which is harder for novice programmers [2]. Additionally, Perry et al.'s. show that users tend to develop misplaced confidence in AI-generated output, which leads to insecure solutions while

users themselves think them to be safe [10]. Jonsson et al.'s. supplement this by presenting how increasing reliance on AI could reduce programmer autonomy [5]. The combination of these observations increases the importance of prompting as the central mechanism that users use in guiding, verifying and interpreting AI output, which directly aligns with our study on how skill levels shape prompting behaviour and user experience in AI-assisted programming.

Despite the contributions of similar studies there exist some gaps that our study seeks to fill. Earlier studies did not use a prompt-based AI like ChatGPT to solve tasks of different difficulty levels, nor did they seek to provide useful insight into how prompting affected solution quality and user experience. Existing studies mostly focus on either a single user group, a single tool, or they focus on one single outcome instead of the combination of user prompts, iterations and how they validated and determined if the AI-generated code was usable. Our study seeks to address these gaps by examining how participant coding level affects prompting, quality of solution and their overall experience using ChatGPT.

3 ChatGPT

To study how programming experience shapes AI-assisted problem solving, we used ChatGPT as the interaction environment[3]. Participants engaged with the model through a chat-based interface, where they could request code, debugging help, or conceptual explanations [9]. Interactions were iterative, allowing users to refine prompts and build on previous responses. ChatGPT thus functioned as a flexible, natural language programming assistant throughout the tasks.

4 Methods

In this section, we will describe the methodological approach used in our study. We will outline the recruitment of participants, the workshop, our data collection, and the analytical procedure applied.

4.1 Participants and recruitment

The target group for this paper is programmers of all levels, including both students and professionals in the industry. The way students use AI will likely influence their future career in the industry. Analyzing their interaction and problem-solving techniques can provide insight into the differences and similarities in AI utilization.

The candidates were recruited through personal networks. Each group member recruited two to four candidates. Diverse experience levels were accounted for, and actively sought out. This way a variety of task-solving approaches could be captured. Solving the tasks required basic knowledge in programming, as well as familiarity with Python. Naturally this excluded complete beginners. Candidates with a minimum of 2 months of programming experience were considered. The final selection was 10 participants, with programming experience from 2 months to 5 years.

4.2 Coding task and questionnaire

Two coding tasks were created from scratch to be solved during the workshop. The first task was to modify existing code, such as fixing bugs and filling in missing code. They had to keep track of multiple files that together formed an "House Manager" app (see appendix A).

The task included both backend python coding and SQL, in addition to frontend HTML. The second task provided the participants more freedom to develop an app in any way they wanted (see appendix A). They were tasked with creating an app that keeps track of groceries and shopping lists. To guide them, certain conditions had to be met, such as “the app should get data from an API”. These tasks were relatively complex, making participants manage interdependent data and code. This way all participants, regardless of level, would be challenged - making it necessary to use AI. The questionnaire was made to gain deeper insight into the participants’ prompting behavior and experiences using AI in coding, providing a more overall picture.

4.3 Workshop

The workshop was conducted in the evening at Media City Bergen. The workshop included a total of 8 participants. 2 participants were unable to attend the workshop. They had individual workshops at a later date with a group member present, following the same workshop plan (table. 1). Each participant was asked to bring their own computer. Light snacks and beverages were served. Before starting the participants signed a consent form (see appendix B), consenting to using their work and chatlogs with chatGPT in the workshop for analysis. They could retract their participation in the project at any point.

To start the workshop, the students introduced the project and the agenda for the workshop, and participants could ask questions. The participants were sent the task descriptions digitally, and had access to it throughout the workshop. The participants logged into pre-made ChatGPT accounts. This way the student had full accessibility to their chat logs. Some participants were not able to log in, and therefore used their own personal accounts.

They were sent a document with two tasks and a questionnaire. When given a signal to begin, they started the first task. They were given 20 minutes to complete task one. Immediately after, they started the second task. This task lasted 40 minutes. In both tasks the participants could use chatGPT as much and in whichever way they wanted.

Proceeding this, they completed a questionnaire. Here they reflected over solution quality, trust in the answers given by the AI, and satisfaction of the solution-method (see appendix C). Before they closed their computer, the students made sure their interaction with ChatGPT was saved. Finally, the workshop was wrapped up, and the participants were thanked graciously for their participation.

4.4 Data Collected

From the workshop we gathered chatlogs, code solutions, and answers to the questionnaire. There were a total of 16 chatlogs. The participants copied the link to their chatlogs and sent them by mail, in addition to pasting it in the questionnaire. Altered code was sent by mail. This gives anyone with the link the ability to access that specific chatlog. No personal details are included, allowing all participants to be anonymous. Ultimately, 18 files were sent.

The participants were timed while solving the coding tasks. If participants were finished before time ran out, they signaled this by raising their hand. This way we had an overview of the amount of time each participant used in relation to the amount of code

Time	Activity	Details
5 min	Ice breaker	- Participants introduced to the project and workshop - Participants could ask questions
20 min	First task	- Code with multiple files creating an app - App that manages houses, participants must manage backend and frontend (dashboard) - Participants filled in missing code and fixed bugs in partially-finished code
40 min	Second task	- Freeform coding - Creating an app that keeps tabs on food in fridge and creates shopping list - Participants must create an app that fulfills certain criteria. Must create backend and frontend
15 min	Questionnaire	- Participants could reflect on prompting behaviour, trust in AI, satisfaction with answers, etc. - Filled it out digitally
5 min	Wrap up	- Concluded the workshop - Thanked the participants

Table 1: Workshop plan

they had completed. To gain insight into prompting behavior and reasoning, the participants completed a Google form questionnaire. From the answers we gathered insight into perceived learning using AI, trust in solutions given, and the experiences using AI.

4.5 Analysis Methods

We chose to conduct a workshop because it gave us the opportunity to observe how programmers of varying experience levels interacted with the AI tool ChatGPT while solving programming tasks. Workshops provide a structured and naturalistic environment where user behavior, tool dependencies, and problem-solving strategies can be observed as they unfold in real time [12]. The fixed task structure made the data easy to compare across participants.

Our analysis combined qualitative and quantitative data sources: chat logs, code submissions and questionnaire responses. Participants were grouped into three sections: novice, intermediate, or experienced, based on self-reported background and knowledge of programming. First, we read all chat logs to gain an overall understanding of how participants used ChatGPT. During this process, we identified recurring prompting patterns, including error-focused prompts, clarification questions, high-level design requests, and role-based instructions such as asking ChatGPT to act as a “full-stack developer”. We also examined how much context participants provided, and whether their interaction style was reactive (responding to errors step-by-step) or proactive (using ChatGPT to plan the

solution). These observations were organised into informal prompting categories, which were then used to compare behaviours across experience levels and task types.

A simple descriptive analysis supported the qualitative findings: for each participant we counted the number of prompts, iteration cycles and instances of role-based prompting, and whether ChatGPTs initial suggestions were accepted or refined by the users. These measures helped identify differences in AI reliance and efficiency.

Solution quality was assessed by reviewing the final code for correctness, completeness and signs of critical engagement with ChatGPTs output, such as testing or modification, and these evaluations were compared across experience groups. Finally, questionnaire responses were used to contextualise the behavioural data. Ratings of trust, perceived learning and satisfaction were summarised descriptively, and open-ended answers were reviewed for recurring themes related to prompting behaviour and AI reliance. These themes were compared with the log and code analyses to triangulate the results.

4.6 Reproducibility

The tasks are included in the appendix, making it easy for future studies to replicate or build on this work. Our hope is to encourage people to be more aware of how they use generative-AI in coding tasks, and to develop prompts that support effective problem solving rather than dependence. We hope further research will investigate how task difficulty, prompting behavior and programming experience shape interaction with AI-tools.

4.7 Ethics

In order to uphold ethical guidelines [11], all participants were anonymised and informed about the purpose of the study and the data collected. Participants signed a consent form, and could withdraw at any point. To avoid collecting personal data through a third-party system, we created multiple temporary accounts. Additionally, simply copying the chat log removes any personal details. This way, participants who used their personal accounts were also anonymised. The study also takes into account the role of the researcher and how the research team's own experience with AI tools can influence the design of the study and the interpretation of the results [11]. To reduce bias, standardized instructions, similar tasks, and consistent facilitation are used. Diversity in participant experience is prioritized to promote fairness and inclusion. The goal is to ensure that the study both meets scientific requirements and safeguards participants' autonomy, privacy, and comfort in interacting with AI systems.

5 Results

All participants completed the two programming tasks using ChatGPT, yielding rich data on prompt strategies, AI reliance, and outcomes. Analysis of the 16 chat transcripts ($n=8 \times 2$ tasks) reveals clear differences in how novice and experienced programmers interacted with ChatGPT. In particular, we observed distinct prompting behaviors by experience level, variations in AI reliance depending on task difficulty, and notable patterns in time investment, iteration counts, and solution quality.

Novice Users – Reactive and Minimal Context Prompts: Novices tended to input very brief prompts, often just error messages or bare task descriptions, and relied on ChatGPT to infer context. For instance, one novice simply pasted a pip error log without any explanation (e.g., `ERROR: Invalid requirement: 'flask=4.0.0'...`) and awaited guidance. ChatGPT correctly interpreted the issue (using `=` instead of `==` in a requirements file) and suggested the fix with no additional context provided by the user. This pattern of providing minimal information was common. Another novice (User B) in the creation task copied the entire task description, prompting ChatGPT to generate the entire solution. Novices also often asked very basic follow-up questions that revealed gaps in understanding (for example, “*hvordan importerer jeg i terminalen?*” – “how do I import in the terminal?”), to which ChatGPT provided fundamental explanations. Overall, novice prompts were reactive, low-context, and focused on single issues, leaving ChatGPT to define the problem and propose solutions.

Experienced Users – Strategic and Context-Rich Prompts: In contrast, experienced participants used structured queries and supplied clear context. They often specified the role ChatGPT should take. For example, one user (User F) prompted ChatGPT to act “*som en profesjonell fullstack-developer*” (“like a professional full-stack developer”) and produce an “*imponerende besvarelse*” with clear step-by-step guidance. This framing prompted ChatGPT to deliver a much more comprehensive response from ChatGPT covering architecture, file structure, and implementation steps. Similarly, another experienced participant prefaced the open-ended task by assigning ChatGPT a role: “*Du er en erfaren techlead...*” (“You are an experienced tech lead...”), effectively asking for high-level guidance rather than just code. Experienced users also shared fuller context, such as the entire codebase and task description in one prompt - rather than isolated errors, enabling ChatGPT to analyze and propose fixes step-by-step. Overall, experienced users treated prompting as a skill, using role instructions and extensive context to steer ChatGPT, unlike novices who relied on ChatGPT to set the agenda.

Post-task questionnaire responses echoed these differences in prompting. Novice participants reported often simply following ChatGPT's suggestions with minimal input of their own, one noted that they “let ChatGPT write all the code” and just did what it instructed. In contrast, experienced participants described more proactive strategies, such as using ChatGPT to quickly generate boilerplate code or set up project structure so they could focus on the more complex aspects of the task. These self-reported strategies reinforce the observed gap in prompting approaches: novices tended to take a passive, reactive role, whereas experienced users actively guided the AI's contributions.

5.1 Task Difficulty and Reliance on AI

Debugging Task (Task 1): The first task (debugging and modifying an existing “House Manager” app) highlighted clear differences in AI reliance. Novices depended on ChatGPT for every basic setup and minor bugs - such as asking how to activate virtual environments, install Flask, or run the script, and then followed the bot's instructions closely. One novice needed multiple exchanges just to resolve simple configuration issues. In contrast, some experienced

participants handled simple issues themselves and used ChatGPT mainly to speed up complex debugging or confirm their reasoning. For instance, an experienced user (User E), quickly recognized that the virtual environment setup was Windows-specific, checked this with ChatGPT, and got the suggestion to recreate it on macOS. In general, for task 1, novices treated ChatGPT as a necessary guide through each error, while experienced users treated it as a time-saving helper.

Open-Ended Creation Task (Task 2): The second task (developing a new app given a set of requirements) was more complex, leading to increased AI reliance across the board. Novices largely outsourced the entire process - design and a large portion of the coding to ChatGPT. One user (User B) provided only the bare list of requirements, and after receiving ChatGPT's initial proposal, simply responded "*ja, og forklar stegene underveis*" ("yes, and explain the steps along the way"), prompting ChatGPT to generate an entire codebase with explanations. Other novices similarly accepted ChatGPT's first proposal and proceeded straight to full implementation. In contrast, experienced participants also relied heavily on ChatGPT, but in a far more interactive and controlling manner. They often made strategic decisions and then asked the AI to adjust or fill in details. For instance, after ChatGPT suggested a React front-end by default, User C requested the front-end be built in Vue instead and specifically asked how to set up a Vue project with Vite. This shows an experienced user leveraging ChatGPT for implementation help while still injecting their own technology preferences. Another experienced participant (User G) used ChatGPT in a collaborative loop - setting up the environment, running the code, then returning to ChatGPT for the next steps or troubleshooting specific issues. Such interactions illustrate that higher-skilled users did not just hand the problem over to the AI; instead, they used ChatGPT as a partner, relying on it for fast solutions to sub-tasks and for verifying approaches, especially given the task's difficulty. In summary, task 2 amplified the skill gap. Novices let ChatGPT design and build the solution end-to-end, while experienced users used it as a collaboration assistant to accelerate execution of their own plans.

Participants' survey feedback corroborated these observed patterns of reliance. Nearly all participants indicated that using ChatGPT made the tasks easier or more manageable. Novices in particular often said they would have struggled to complete the challenges without the AI's help - for example, one respondent admitted, "*I don't know how I would have completed the tasks without it.*" Experienced users likewise found the AI very helpful, but they tended to frame it as a convenience or time-saver rather than a crutch. One experienced participant mentioned that ChatGPT "*streamlined the process*" of development, allowing them to set up the project faster and handle unfamiliar details (such as framework setup) more efficiently. Notably, only a single participant felt that using AI made the work *harder*, explaining that ChatGPT's solution was too convoluted and difficult to follow. Overall, these reflections underscore that novices viewed ChatGPT as an indispensable guide to get through difficult tasks, whereas experienced users saw it as a useful accelerator and support tool for solving complex tasks more efficiently.

5.2 Time, Iterations, and Solution Quality

Time and Prompt Iterations: Novices required more prompt iterations, addressing one error at a time in a trial-and-error manner. They would fix one error with ChatGPT's help, then encounter the next issue and ask again, in a chain of small steps. For example, one novice (User B) sequentially resolved installation and import issues through multiple AI interactions. This piecemeal approach meant novices spent a significant portion of the session cycling through errors one by one. In contrast, experienced users used broader prompts that often solved problems at once and asked follow-ups to extend or refine solutions rather than just fix errors. As noted above, one participant provided the entire broken code and obtained a comprehensive fixed version in one go, avoiding a long series of micro-questions. This highlights an efficiency gap: novices spent more time in incremental debugging, while experienced users accomplished tasks with fewer, more strategic interactions.

Final Solution Quality. All users had a solution produced with substantial AI assistance, but the robustness and completeness of these solutions appeared to differ. Novices typically accepted ChatGPT's first draft with minimal modification. The AI often generated syntactically correct and satisfied the stated requirements - such as a fully repaired Flask app with fixes clearly annotated. The final code given by ChatGPT was high-quality on the surface. However, novices rarely tested or refined it, sometimes overlooking TODOs or missing elements that ChatGPT explicitly flagged. For example, after providing the fixed Flask code, it noted that the HTML template still needed an image tag for the /house route. In the chat logs, novices did not follow up on such details, suggesting that they accepted the solution as-is without addressing these finishing touches. Experienced users, however, were more likely to run the code, identify runtime or configuration issues, and return to ChatGPT for targeted fixes. For example, User G, after generating a Node/Express server and a simple frontend, attempted to start the app and discovered a missing package.json (the project configuration file). This was brought back to ChatGPT, which then guided them to properly initialize the Node project (`npm init -y`) to create the file. This iterative testing led to more complete, executable solutions. Some even extended the baseline requirements. For instance, one user asked ChatGPT to incorporate cloud deployment (using a Supabase backend and Vercel hosting) after the initial solution was working, indicating an attempt to improve the solution's real-world viability. Novice users generally did not reach this stage; they focused on getting the core functionality working and tended to stick with ChatGPT's first complete solution.

In summary, while all participants achieved functional solutions with ChatGPT, the journey and end results differed. Novices, through numerous micro-iterations, arrived at solutions largely authored by the AI and took them at face value. This unquestioning acceptance was reflected in their self-reports: one novice admitted "*I just assumed the answer was correct, it looked professional.*" In contrast, experienced participants, with fewer but more targeted prompts, treated ChatGPT's output as a draft - testing it, tweaking it, and sometimes extending it. The result was that experienced users' final code was often more aligned with best practices (by virtue of custom choices like technology stack or deployment) and more rigorously validated, whereas novice solutions were directly

AI-generated and correct with respect to the prompt, but not necessarily vetted beyond that. Participants were divided on how using ChatGPT affected their learning. Roughly half of the group felt that the AI helped them learn more – often by quickly clarifying errors or demonstrating how to implement solutions – whereas the others believed they would have learned more by solving the tasks on their own without AI, since ChatGPT had effectively done the hardest parts for them. Notably, this split in perceived learning did not strictly follow experience level: a few novices recognized that they learned less with heavy AI assistance, while some experienced users appreciated that ChatGPT exposed them to new approaches or best practices they might not have thought of themselves. In other words, although ChatGPT enabled everyone to complete the tasks, its impact on individuals’ learning varied. Overall, these findings reflect how programming experience and task difficulty together shaped not just the process of obtaining answers from an AI, but also the quality and reliability of the solutions – and even participants’ own learning experience.

6 Discussion

The findings of this study provide insight into how programmers use AI based on skill level and task difficulty. A key contribution of this project is highlighting how prior programming experience does not simply influence solution correctness [8], but also affects the role AI plays in the problem-solving process.

6.1 Task Difficulty and Prompting Skills

Task difficulty strongly influences how people engage with AI. Complex tasks tend to push users towards more iterative, exploratory and conversational use of AI, unlike simpler tasks where such behavior was limited. This suggests that AI should adapt its level of support dynamically based on a user’s performance rather than relying on fixed categories like “novice” or “expert”. At the same time, prompting itself is emerging as a key programming skill. Good prompting involves refining instructions, evaluating AI output, and clearly communicating intent. Expert programmers tend to engage in this process intuitively, whereas novices often lack the metacognitive skills to do so. Together, these findings highlight the need for AI systems to provide gentle, supportive guidance that helps users develop strong prompting habits while still maintaining their autonomy.

6.2 Implications for the Future of Programming

Together, these findings point to several broader implications for the future of programming:

- AI will increasingly become part of the learning process, not just an efficiency tool, and should therefore offer explanations and reflection opportunities.
- The role of the programmer may shift toward evaluating, modifying, and understanding AI-generated code, particularly for novice learning through AI.
- Designing AI systems that provide support while still allowing users to think and make decisions themselves. This balance helps prevent users from becoming dependent on AI and encourages the development of their own programming skills.

- Future AI programming interfaces may benefit from adaptive scaffolding, adjusting their behavior based on task complexity and user prompting patterns.

In short, AI-assisted programming is not a single, uniform activity; it varies depending on a person’s experience, the task difficulty, and how they interact with the AI.

7 Limitations and future work/improvements

The workshop included a total of 8 participants, in addition to two participants at a later date. Due to limited recruitment and eligibility requirements, the data gathered is therefore limited, and would have been richer with more participants. Moreover, the difficulty of the tasks may also have influenced the results. The tasks were fairly difficult for the novices, requiring a higher level of understanding. Some participants might therefore have experienced mental fatigue, potentially influencing prompting behavior and self-sufficiency. We used python and a frontend language of choice in this study. How programmers use AI with other coding languages remains unexplored. Furthermore, the predefined tasks do not fully reflect real-world programming conditions, that may vary in complexity, collaboration and pressure.

Future research should aim to include more participants, preferably from diverse backgrounds. Most of the participants were either students at UiB or working in the industry. It should also incorporate a wider range of tasks across different coding languages to capture varied skills and requirements. Additionally, the analysis should take place over a longer period of time to capture programming in a more natural, real-world setting – something we could not achieve due to time and resource constraints.

8 Conclusion

This study examines how programmers of different skill levels use AI when solving coding tasks, focusing on AI-reliance, prompting behavior, solution quality, and user experience. Eight participants took part in a workshop, in addition to two more at a later date. They completed two programming tasks with ChatGPT, and then a questionnaire about their reasoning and learning outcomes. The results show a clear difference between novices and experienced programmers. Novices relied heavily on ChatGPT, asking many narrow prompts, and often accepted generated code with little revision. In contrast, experienced programmers used fewer, more strategic prompts, employed role or full-context instructions, and treated ChatGPT as a debugging or productivity tool. They were also more likely to refine and improve the AI’s output.

Findings suggest that AI use is closely tied to task difficulty and skill level, with novice and experienced programmers utilizing AI in significantly different ways. As AI-assisted coding becomes standard, programmers must develop strong prompting skills and the ability to critically evaluate generated code. LLMs should adjust their guidance based on user skill and task demands, offering explanations and reflective prompts to support learning. Futurework should explore adaptive interface designs and their impact on users’ learning and satisfaction as programming practices continue to evolve.

References

- [1] Manal Alanazi, Ben Soh, Halima Samra, and Alice Li. 2025. The Influence of Artificial Intelligence Tools on Learning Outcomes in Computer Programming: A Systematic Review and Meta-Analysis. *Computers* 14, 5 (2025). doi:10.3390/computers14050185
- [2] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 78 (April 2023), 27 pages. doi:10.1145/3586030
- [3] Ivana Belcic and Chris Stryker. n.d.. What is GPT (generative pretrained transformer)? <https://www.ibm.com/think/topics/gpt>. Retrieved 2025-11-15.
- [4] Alessio Bucaioni, Hampus Ekedahl, Vilma Helander, and Phuong T. Nguyen. 2024. Programming with ChatGPT: How far can we go? *Machine Learning with Applications* 15 (2024), 100526. doi:10.1016/j.mlwa.2024.100526
- [5] Martin Jonsson, Jakob Tholander, Mattias Rost, Advait Sarkar, Ylva Fernaeus, Ahmed Kharrufa, and Thomas Ludwig. 2025. The End of Programming as We Know It: Envisioning Radical Re-Conceptualizations of Co-Coding with AI. In *Adjunct Proceedings of the Sixth Decennial Aarhus Conference: Computing X Crisis (AAR Adjunct '25)*. Association for Computing Machinery, New York, NY, USA, Article 30, 6 pages. doi:10.1145/3737609.3747093
- [6] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 455, 23 pages. doi:10.1145/3544548.3580919
- [7] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. 2024. How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research* (Koli, Finland) (Koli Calling '23). Association for Computing Machinery, New York, NY, USA, Article 3, 12 pages. doi:10.1145/3631802.3631806
- [8] Lenneke Kuijter and Elisa Giaccardi. 2018. Co-performance: Conceptualizing the Role of Artificial Agency in the Design of Everyday Life. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) (CHI '18). Association for Computing Machinery, New York, NY, USA, 1–13. doi:10.1145/3173574.3173699
- [9] OpenAI. n.d.. ChatGPT Capabilities Overview. <https://help.openai.com/en/articles/9260256-chatgpt-capabilities-overview>. Accessed: 2025-11-21.
- [10] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) (CCS '23). Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157
- [11] Yvonne Rogers, Helen Sharp, and Jennifer Preece. 2023. *Interaction Design: Beyond Human-Computer Interaction* (6 ed.). Wiley.
- [12] Katja Thoring, Rüdiger M. Müller, and Petra Badke-Schaub. 2020. Workshops as a Research Method: Guidelines for Designing and Evaluating Artifacts Through Workshops. In *Proceedings of the 53rd Hawaii International Conference on System Sciences*. <https://scholarspace.manoa.hawaii.edu/server/api/core/bitstreams/c78f65c6-8097-4099-a2a5-09b384c17e69/content>

9 Appendix

9.1 Appendix A - Task Description

Oppgave 1: Fiks kode.

[[Link to code](#)] Du får utdelt en delvis fungerende “House Manager” app. Her er det flere deler av koden som ikke fungerer/mangler. Fiks og fyll ut manglende kode. Mesteparten av koden finner du i `functions.py` og `app.py`. Din oppgave er å få appen til å kjøre og rette feilene ved hjelp av AI. Du kan bruke AI til å stille spørsmål, få forklaringer, forslag til feilsøking eller kodeendringer og selv velge hvilke forslag du vil følge. Når du får programmet til å kjøre må du sjekke at all funksjonalitet fungerer.

Du skal bruke virtual environmentet som ligger i mappen, så du må huske å aktivere det. Hvis du får tid, kan du også utvide appen med funksjonalitet eller bedre design av grensesnittet. Målet er ikke å lage en perfekt løsning, men å observere hvordan du bruker AI til å forstå og reparere et eksisterende prosjekt. 20 min

Oppgave 2: Matlager & Handleliste (v.2.1)

Lag en app som hjelper brukeren å holde oversikt over matvarene de har hjemme og å lage en handleliste. Du står fritt til å velge teknologi og arkitektur, men løsningen skal inkludere følgende:

Krav til løsning.

- **Backend:** Skal håndtere kommunikasjon med databasen og eksterne API-er.
- **Frontend / Brukergrensesnitt:** Valgfritt rammeverk. Skal gi en oversikt over matlager og handleliste.
- **Database:** Skal lagre informasjon om matvarer brukeren har hjemme.
- **Integrasjon mot API:** Bruk data fra `matvaretabellen.no`. Rens og bruk kun relevant informasjon fra API-et.

Funksjonelle krav.

- Appen skal hente informasjon om matvarer fra API-et.
- Matvarer som legges inn i databasen representerer varer brukeren har hjemme.
- Dersom en vare finnes i databasen, skal den fjernes fra handlelisten (markerer som kjøpt).
- Appen skal fungere på mobil
 - Brukergrensesnittet må tilpasse seg mindre skjermer slik at alle funksjoner er lett tilgjengelige.
 - Løsningen skal testes på både desktop og mobil.

Mål. Målet er ikke å lage en perfekt app, men å vise hvordan du:

- Bruker **AI** til å planlegge, forstå oppgaven, ta valg og implementere løsningen,
- Benytter **AI** aktivt i koding, design og utforsking av funksjonalitet,
- Reflekterer over hvordan **AI** bidrar i utviklingsprosessen.

Eksempel på brukerflyt.

- (1) Bruker søker etter “melk”.
- (2) Velger riktig treff fra API-et og legger varen til handlelisten.
- (3) Etter kjøp: trykker “Marker som kjøpt” i appen.
- (4) Varen flyttes til matlageret og fjernes fra handlelisten.
 - **Matlager:** viser matvarene brukeren har hjemme.
 - **Handleliste:** viser varene som fortsatt mangler.

9.2 Appendix B - Consent Form

Request for participation in a research project

“How does task difficulty and prior programming experience influence solution quality, time, and prompting behaviour when users solve programming tasks with an AI chat interface?”

Background and Purpose

We are conducting a research study to understand how programmers interact with AI while coding. This study is part of “Research Themes in Human-Machine Interaction”, where our group has chosen to investigate how task difficulty and programming experience affect the way people use the AI-tool ChatGPT for programming. AI is becoming increasingly integrated into programming, and therefore it is important to understand how these tools affect learning, problem-solving strategies and code quality. Your participation will help us gain insight into how people with different coding skills choose to use ChatGPT as a tool to solve problems.

Practical information

Time: 60 minutes

Date: 29. October, 16:00

Location: Media City Bergen, Seminarrom 2

Refreshments will be served during the session.

Your tasks will be

- Solving 2–3 programming tasks in Python using ChatGPT.
- Give us access to your prompts afterwards, or screen record your session.
- Reflect along the way as you do the tasks, what will be your next prompt and why?
- Participate in a short written interview (5–10 questions) after completing the tasks.

Consent for Data Collection and Use

☐ I agree that my AI prompts, code solutions, reflections, and interview responses can be recorded (via screen recording, chat logs, or other forms of documentation) and used in this research.

Voluntary participation

Participation in this study is completely voluntary. You can withdraw at any time without giving any reason. If you withdraw, all data about you will be deleted.

What will happen to the information about you?

All personal data will be treated confidentially and in accordance with GDPR regulations, and your data will be anonymized in all reports and publications.

Consent for participation in the study

I have received information about the project and am willing to participate.

(Signed by participant, date)

9.3 Appendix C - Questionnaire

[[Link to answers](#)]

- (1) Hvor lenge har du programmert / hvor mye erfaring har du med programmering? (Svar kort)
- (2) Har du erfaring med andre programmeringsspråk enn Python? (Kan minst skrive kode og løse enkle oppgaver). Hvis ja, hvilke?
- (3) Brukte du AI?
- (4) Hvorfor / hvorfor ikke?
- (5) Hva var din strategi for å løse oppgavene? Svar i korte setninger.
- (6) Hvilken type hjelp forventet du at AI skulle gi deg? Dersom du ikke brukte AI, svar "brukte ikke AI".
- (7) Endret du måten du brukte AI underveis. Hvorfor / hvorfor ikke? Dersom du ikke brukte AI, svar "brukte ikke AI".
- (8) Hvor mye stolte du på svarene fra AI? Dersom du ikke brukte AI, svar ut i fra tilliten du har til AI generelt.
- (9) Hvordan vurderte du om svaret var korrekt eller nyttig? Dersom du ikke brukte AI, svarte ut i fra hvordan du vurderer svarene til AI generelt.

- (10) Opplevde du at AI gjorde oppgaven enklere, vanskeligere, eller bare annerledes? Dersom du ikke brukte AI, svarte ut i fra hvordan du synes AI endrer oppgavene generelt.
- (11) Hva synes du fungerte bra med å bruke AI i oppgavene? Dersom du ikke brukte AI, svar "brukte ikke AI".
- (12) Hva fungerte dårlig?
- (13) Tror du at du lærte mer eller mindre av å bruke AI sammenlignet med å løse oppgavene selv?
- (14) Dersom du ikke brukte AI, tror du at du hadde lært mer eller mindre dersom du hadde brukt AI?
- (15) Hvordan kunne AI-verktøy vært forbedret for å hjelpe deg bedre?
- (16) Dersom du ikke brukte AI: hvordan kunne AI-verktøy vært forbedret slik at du ønsker å bruke det?
- (17) Dersom du brukte din egen ChatGPT konto: vennligst eksportert alle chattene dine ("del" i øvre høyre hjørne)
- (18) Her kan du komme med noen tilbakemeldinger til studentene/prosjektet? (Ikke obligatorisk)

9.4 Appendix D - Code

[[Storage for Code](#)]